

1. Algorithm

Main Algorithm

1. Start.
2. Declare `choice`.
3. Display the menu:
 1. Right-aligned increasing triangle
 2. Right-aligned increasing number triangle
 3. Right-aligned decreasing triangle
 4. Exit
4. Read `choice`.
5. Use a `switch` statement:
 - If `choice` = 1, call `increasingTriangle()`.
 - If `choice` = 2, call `increasingNumberTriangle()`.
 - If `choice` = 3, call `decreasingTriangle()`.
 - If `choice` = 4, terminate the program.
 - Otherwise, display "Invalid choice."
6. Repeat the menu until the user selects Exit.
7. Stop.

Algorithm: Right-aligned Increasing Triangle

1. Start.
 2. Input the number of rows `n`.
 3. If `n` <= 0, display an error message and return.
 4. For `i` = 1 to `n`:
 - Print `n - i` groups of spaces.
 - Print `i` asterisks.
 - Move to the next line.
 5. Stop.
- For `n` = 4:

```
      *
     * *
    * * *
   * * * *
```

Algorithm: Right-aligned Increasing Number Triangle

1. Start.
2. Input the number of rows `n`.
3. If `n` <= 0, display an error message and return.
4. For `i` = 1 to `n`:
 - Print `n - i` groups of spaces.
 - Print the natural numbers from 1 to `i`.
 - Move to the next line.

5. Stop.
For n = 4:

```

    1
  1 2
1 2 3
1 2 3 4

```

Algorithm: Right-aligned Decreasing Triangle

1. Start.
2. Input the number of rows n.
3. If n <= 0, display an error message and return.
4. For i = n down to 1:
 ◦ Print n - i groups of spaces.
 ◦ Print i asterisks.
 ◦ Move to the next line.
5. Stop.
For n = 4:

```

* * * *
 * * *
  * *
   *

```

2. Test-Case Table

Test Case	Menu Choice	Input n	Expected Result	Actual Result	Status
1	1	4	4-row right-aligned increasing * triangle	As expected	Pass
2	2	4	Right-aligned rows 1, 1 2, 1 2 3, 1 2 3 4	As expected	Pass
3	3	4	Right-aligned decreasing * triangle from 4 to	As expected	Pass
4	1	1	One *	As expected	Pass
5	2	1	One 1	As expected	Pass
6	3	1	One *	As expected	Pass
7	1	0	Error: number of rows must be positive	As expected	Pass
8	2	-3	Error: number of rows must be positive	As expected	Pass
9	7	—	Invalid menu choice	As expected	Pass
10	4	—	Program terminates	As expected	Pass

3. C Program

```
#include <stdio.h>

void increasingTriangle(void);
void increasingNumberTriangle(void);
void decreasingTriangle(void);

int main(void)
{
    int choice;

    do
    {
        printf("\nRIGHT-ALIGNED PATTERNS\n");
        printf("1. Right-aligned increasing triangle\n");
        printf("2. Right-aligned increasing number\n");
triangle\n");
        printf("3. Right-aligned decreasing triangle\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                increasingTriangle();
                break;

            case 2:
                increasingNumberTriangle();
                break;

            case 3:
                decreasingTriangle();
                break;

            case 4:
                printf("Exiting program...\n");
                break;

            default:
```

```
        printf("Invalid choice. Please enter 1 to  
4.\n");  
    }
```

```
    } while (choice != 4);
```

```
    return 0;
```

```
}
```

```
/* Right-aligned increasing triangle */
```

```
void increasingTriangle(void)
```

```
{
```

```
    int n, i, j;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &n);
```

```
    if (n <= 0)
```

```
    {
```

```
        printf("Number of rows must be positive.\n");
```

```
        return;
```

```
    }
```

```
    for (i = 1; i <= n; i++)
```

```
    {
```

```
        /* Print leading spaces */
```

```
        for (j = 1; j <= n - i; j++)
```

```
        {
```

```
            printf("  ");
```

```
        }
```

```
        /* Print asterisks */
```

```
        for (j = 1; j <= i; j++)
```

```
        {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
/* Right-aligned increasing number triangle */
```

```
void increasingNumberTriangle(void)
```

```
{
```

```
    int n, i, j;
```

```

printf("Enter the number of rows: ");
scanf("%d", &n);

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = 1; i <= n; i++)
{
    /* Print leading spaces */
    for (j = 1; j <= n - i; j++)
    {
        printf("  ");
    }

    /* Print natural numbers */
    for (j = 1; j <= i; j++)
    {
        printf("%d ", j);
    }

    printf("\n");
}
}

/* Right-aligned decreasing triangle */
void decreasingTriangle(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        /* Print leading spaces */

```

```

        for (j = 1; j <= n - i; j++)
        {
            printf("  ");
        }

        /* Print asterisks */
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }

        printf("\n");
    }
}

```

4. Sample Execution

Test 1 — Right-aligned Increasing Triangle

```

Last login: Fri Sep 18 22:08:00 on ttys000
[bhavya@Bhavyas-MacBook-Air ~ % nano day26.cpp
[bhavya@Bhavyas-MacBook-Air ~ % clang day26.cpp -o day26
[bhavya@Bhavyas-MacBook-Air ~ % ./day26

```

RIGHT-ALIGNED PATTERNS

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice: 1

Enter the number of rows: 4

```

      *
     * *
    * * *
   * * * *

```

RIGHT-ALIGNED PATTERNS

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Test 2 — Right-aligned Increasing Number Triangle

```
Enter your choice: 2
Enter the number of rows: 4
  1
 1 2
1 2 3
1 2 3 4

RIGHT-ALIGNED PATTERNS
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 2/
```

Test 3 — Right-aligned Decreasing Triangle

```
Enter your choice:
3
Enter the number of rows: 4
* * * *
 * * *
  * *
   *

RIGHT-ALIGNED PATTERNS
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 4/
```

Invalid Input

```
1. Exit
Enter your choice: 1
Enter the number of rows: 0
Number of rows must be positive.

RIGHT-ALIGNED PATTERNS
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice: 6
```

Invalid Menu Choice

```
Enter your choice: 6
Invalid choice. Please enter 1 to 4.

RIGHT-ALIGNED PATTERNS
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
```

Exit

```
Enter your choice: 4
Exiting program...
bhavya@Bhavyas-MacBook-Air ~ %
```

Two spaces are printed for each missing pattern element (`printf(" ");`). This is important because each displayed asterisk or single-digit number occupies approximately two character positions ("`*` " or "`%d` "), producing the required **right alignment**.